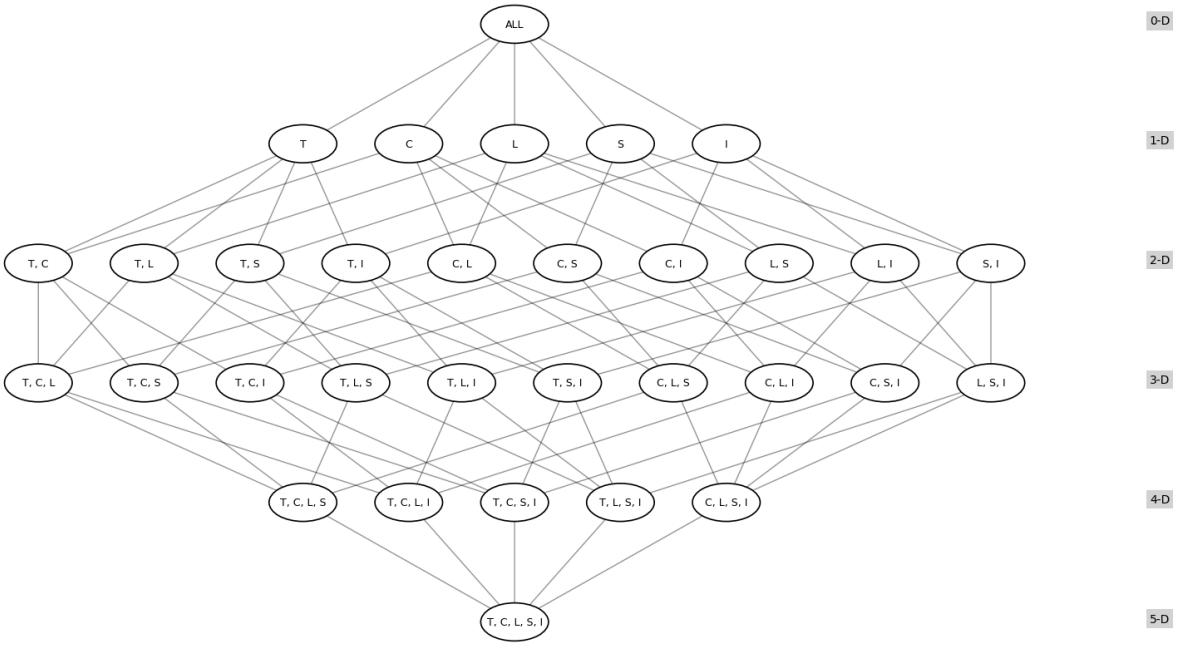
DWBI Assignment 2

Question 1:

P1: Total number of cuboids: 2^n where n is the number of dimensions.Here n = 5 {T,C,L,S,I}

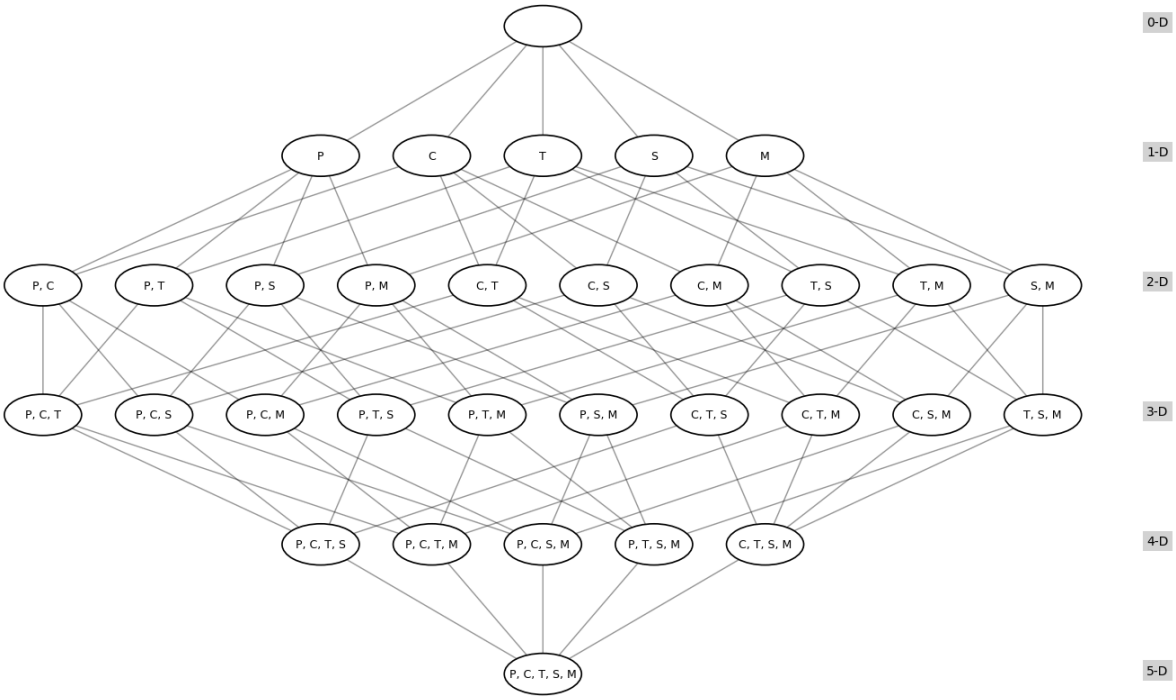
and so the total number of cuboids is 2 ^ 5 which is equivalent to 32.

P2:



Question 2:

P1:



P2:

```
drop database if exists dwbi_a2;
-- Create the database if it doesn't exist
create database if not exists dwbi_a2;
use dwbi_a2;

-- Time dimension table
create table dim_time
(
   time_id int primary key,
   date_key date not null,
```

```
    day_name varchar(10),
    month_number int not null,
    quarter_number int not null,
    year_number int not null
);

-- Product dimension table
create table dim_product
(
    product_id int primary key,
    product_name varchar(100) not null,
    category varchar(50) not null,
    department varchar(50) not null,
    brand varchar(50),
    price decimal(10, 2)
);

-- Customer dimension table
create table dim_customer
(
    customer_id int primary key,
    customer_name varchar(100) not null,
    email varchar(100) unique,
    city varchar(50) not null,
    country varchar(50) not null,
    customer_segment varchar(50) not null
);

-- Store dimension table
create table dim_store
(
    store_id int primary key,
    store_name varchar(100) not null,
    city varchar(50) not null,
    region varchar(50) not null,
    country varchar(50) not null,
    store_type varchar(50)
);
```

```
-- Payment method dimension table
create table dim_payment_method
(
   payment_id int primary key,
   payment_type varchar(50) not null,
   provider varchar(50),
   payment_category varchar(50)
);

-- Fact table for sales transactions
create table fact_sales
(
   transaction_id int primary key,
   time_id int not null,
   product_id int not null,
   customer_id int not null,
   store_id int not null,
   payment_id int not null,
   sales_amount decimal(10, 2) not null,
   quantity_sold int not null,
   cost_of_goods_sold decimal(10, 2) not null,
   -- Automatically calculate profit
   profit decimal(10, 2) generated always as (sales_amount - cost_of_goods_sold) stored,
   -- Define foreign key relationships to dimension tables
   foreign key (time_id) references dim_time(time_id),
   foreign key (product_id) references dim_product(product_id),
   foreign key (customer_id) references dim_customer(customer_id),
   foreign key (store_id) references dim_store(store_id),
   foreign key (payment_id) references dim_payment_method(payment_id)
);
```

P3:

```
-- Insert sample data into time dimension
insert into dim_time (time_id, date_key, month_number, quarter_number, year_number,
day_name) values
(1, '2024-01-15', 1, 1, 2024, 'Monday'),
```

```
(2, '2024-01-16', 1, 1, 2024, 'Tuesday'),
(3, '2024-01-17', 1, 1, 2024, 'Wednesday'),
(4, '2024-02-10', 2, 1, 2024, 'Saturday'),
(5, '2024-03-20', 3, 1, 2024, 'Wednesday');

-- Insert sample data into product dimension
insert into dim_product (product_id, product_name, category, department, brand, price) values
(1, 'iPhone 15', 'Electronics', 'Tech', 'Apple', 1099.99),
(2, 'Samsung Galaxy', 'Electronics', 'Tech', 'Samsung', 899.99),
(3, 'Office Desk', 'Furniture', 'Home', 'IKEA', 299.99),
(4, 'Gaming Chair', 'Furniture', 'Home', 'Racer', 199.99),
(5, 'Coffee Maker', 'Home Appliances', 'Home', 'Philips', 149.99);

-- Insert sample data into customer dimension
insert into dim_customer (customer_id, customer_name, email, country, city,
customer_segment) values
(1, 'Alice Johnson', 'alice@email.com', 'USA', 'New York', 'High-Value'),
(2, 'Bob Smith', 'bob@email.com', 'UK', 'London', 'Standard'),
(3, 'Carlos Ruiz', 'carlos@email.com', 'Spain', 'Madrid', 'Standard'),
(4, 'Wei Zhang', 'wei@email.com', 'China', 'Beijing', 'High-Value'),
(5, 'Emma Wilson', 'emma@email.com', 'Australia', 'Sydney', 'Standard');

-- Insert sample data into store dimension
insert into dim_store (store_id, store_name, city, region, country, store_type) values
(1, 'NYC Flagship', 'New York', 'North America', 'USA', 'Premium'),
(2, 'London Central', 'London', 'Europe', 'UK', 'Standard'),
(3, 'Madrid Plaza', 'Madrid', 'Europe', 'Spain', 'Premium'),
(4, 'Beijing Mall', 'Beijing', 'Asia', 'China', 'Standard'),
(5, 'Sydney Opera', 'Sydney', 'Australia', 'Australia', 'Premium');

-- Insert sample data into payment method dimension
insert into dim_payment_method (payment_id, payment_type, provider, payment_category)
values
(1, 'Credit Card', 'Visa', 'Card'),
(2, 'Digital Wallet', 'PayPal', 'Digital'),
(3, 'Bank Transfer', 'Wire', 'Transfer'),
(4, 'Credit Card', 'MasterCard', 'Card'),
(5, 'Buy Now Pay Later', 'Klarna', 'Credit');
```

```
-- Insert sample data into fact table
insert into fact_sales (transaction_id, time_id, product_id, customer_id, store_id, payment_id,
sales_amount, quantity_sold, cost_of_goods_sold) values
(1, 1, 1, 1, 1, 1, 1099.99, 1, 769.99),
(2, 2, 4, 2, 2, 2, 199.99, 1, 139.99),
(3, 3, 3, 3, 3, 3, 299.99, 1, 209.99),
(4, 4, 2, 4, 4, 4, 899.99, 1, 629.99),
(5, 5, 5, 5, 5, 5, 149.99, 1, 104.99),
(6, 1, 2, 1, 1, 4, 1799.98, 2, 1259.98),
(7, 2, 1, 5, 2, 1, 1099.99, 1, 769.99),
(8, 3, 5, 3, 3, 2, 449.97, 3, 314.97),
(9, 4, 3, 4, 4, 3, 299.99, 1, 209.99),
(10, 5, 4, 2, 5, 5, 399.98, 2, 279.98);
```

P4:

```
-- 1. Total sales for all Electronics products
select
   p.category,
   sum(f.sales_amount) as total_sales
from fact_sales f
join dim_product p on f.product_id = p.product_id
where p.category = 'Electronics'
group by p.category with rollup;

-- 2. Sales for Premium stores in Europe using Credit Card payments (no data for this query)
select
   s.store_name,
   s.region,
   s.store_type,
   p.payment_type,
   sum(f.sales_amount) as total_sales
from fact_sales f
join dim_store s on f.store_id = s.store_id
join dim_payment_method p on f.payment_id = p.payment_id
where s.region = 'Europe' and s.store_type = 'Premium' and p.payment_type = 'Credit Card'
group by s.store_name, s.region, s.store_type, p.payment_type with rollup;
```

-- 3. Sales summary by Category, Brand, and total

```
select
   p.category,
   p.brand,
   sum(f.sales_amount) as total_sales
from fact_sales f
join dim_product p on f.product_id = p.product_id
group by p.category, p.brand with rollup;
```

-- 4. Drill-down: Category to individual Electronics products

```
select
   p.category,
   p.product_name,
   sum(f.sales_amount) as total_sales
from fact_sales f
join dim_product p on f.product_id = p.product_id
where p.category = 'Electronics'
group by p.category, p.product_name with rollup;
```

-- 5. Pivot table: Product Category vs Payment Type

```
select
   p.category,
   sum(case when pm.payment_type = 'Credit Card' then f.sales_amount else 0 end) as
credit_card_sales,
   sum(case when pm.payment_type = 'Digital Wallet' then f.sales_amount else 0 end) as
digital_wallet_sales,
   sum(case when pm.payment_type = 'Bank Transfer' then f.sales_amount else 0 end) as
bank_transfer_sales,
   sum(case when pm.payment_type = 'Buy Now Pay Later' then f.sales_amount else 0 end) as
bnpl_sales,
   sum(f.sales_amount) as total_sales
from fact_sales f
join dim_product p on f.product_id = p.product_id
join dim_payment_method pm on f.payment_id = pm.payment_id
group by p.category with rollup;
```

-- 6. All combinations of Customer Country, Store Region, and Product Category

```
select c.country, s.region, p.category, sum(f.sales_amount) as total_sales
from fact_sales f
join dim_customer c on f.customer_id = c.customer_id
join dim_store s on f.store_id = s.store_id
join dim_product p on f.product_id = p.product_id
group by c.country, s.region, p.category

union all
-- Aggregated by Country and Region
select c.country, s.region, null as category, sum(f.sales_amount)
from fact_sales f
join dim_customer c on f.customer_id = c.customer_id
join dim_store s on f.store_id = s.store_id
join dim_product p on f.product_id = p.product_id
group by c.country, s.region

union all
-- Aggregated by Country only
select c.country, null as region, null as category, sum(f.sales_amount)
from fact_sales f
join dim_customer c on f.customer_id = c.customer_id
join dim_store s on f.store_id = s.store_id
join dim_product p on f.product_id = p.product_id
group by c.country

union all
-- Grand total
select null as country, null as region, null as category, sum(f.sales_amount)
from fact_sales f
join dim_customer c on f.customer_id = c.customer_id
join dim_store s on f.store_id = s.store_id
join dim_product p on f.product_id = p.product_id;

-- 7. Compare total sales between January and February 2024
select
  t.month_number,
  sum(f.sales_amount) as total_sales
from fact_sales f
```

```
join dim_time t on f.time_id = t.time_id
where t.year_number = 2024 and t.month_number in (1, 2)
group by t.month_number with rollup;

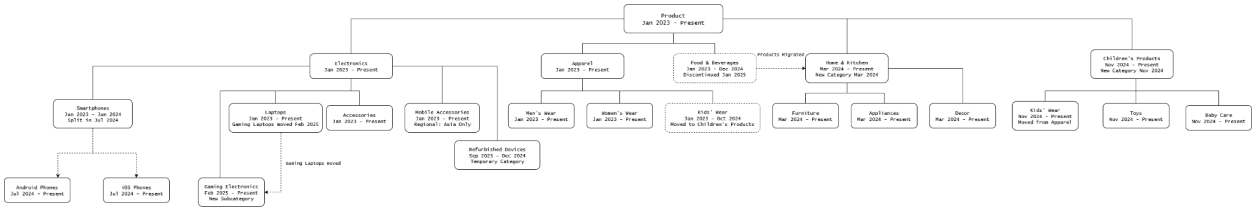
-- 8. Top 3 products by total sales
select
   p.product_name,
   sum(f.sales_amount) as total_sales
from fact_sales f
join dim_product p on f.product_id = p.product_id
group by p.product_name
order by total_sales desc
limit 3;
```

Q3:

A. Classification Hierarchy Evolution Mapping

Temporal Classification Hierarchy Tree (2023 – 2025)

Product Dimension (valid from 2023-01-01 ⟶ present)



Regional Variation: Asian markets include an extra subcategory *Mobile Accessories* under Electronics (from 2023-01-01 onwards).

Temporary Classification: Refurbished Devices (2023-09 ⟶ 2024-12).

Discontinued Category: Food & Beverages (ended 2025-01 ⟶ migrated to Home & Kitchen).

B. Advanced Validity Matrix Construction

Classification Name	Level	Valid From	Valid To	Region	Business Rule/Notes	Migration Path	Query Routing Guideline	Integrity Constraint
Electronics	Subcat	2023-01-01	-	Global	Base Category	-	Use for all periods	One active cat per product
Smartphones	Subcat	2023-01-01	2024-07-01	Global	Pre-split phones	->Android + iOS (2024-07)	Use for sales before Jul 2024	Non-overlap with new subs
Android Phone	Subcat	2024-07-01	-	Global	New split	<-Smartphones	Use for sales after Jul 2024	Split mapping logged
iOS Phones	Subcat	2024-07-01	-	Global	New Split	<-Smartphones	Use for sales after Jul 2024	Split mapping logged
Refurbished Devices	Subcat	2023-09-01	2024-12-31	Global	Temporary	-	Include only in valid interval	Must expire cleanly
Laptops	Subcat	2023-01-01	2025-02-01	Global	Pre-reorg	->Gaming Electronics (2025-02)	Use before Feb 2025	Non-overlap
Gaming Electronics	Subcat	2025-02-01	-	Global	New reorg	<-Gaming Laptops	Use after Feb 2025	Mapping complete
Kids’ Wear	Subcat	2023-01-01	2024-11-01	Global	Moved out of apparel	->Children’s Product	Use before Nov	Gap check

						s	2024	
Children's Products	Category	2024-11-01	-	Global	New Category	<-Kids' Wear	Use after Nov 2024	Migration complete
Food & Beverages	Category	2023-01-01	2025-01-01	Global	Discontinued	-> Home & Kitchen	Use before Jan 2025	No dangling products
Home & Kitchen	Category	2024-03-01	-	Global	Added 2024	<- Food & Beverages (2025)	Use after Mar 2024	Overlap control
Mobile Accessories	Subcat	2023-01-01	-	Asia	Regional only	-	Include only Asia Queries	Regional scope enforced
Sustainable Products Flag	Attribute	2025-06-01	-	Global	Cross-category flag	-	Join by validity interval	Independent intervals

Query Routing Rules

1. **Temporal Validity** – Join sales with classification history where sale_date BETWEEN valid_from AND valid_to.

2. **Regional Validity** – Filter by region = 'Asia' to include regional subcategories (e.g., Mobile Accessories).

3. **Business Rule Validity** – Include temporary subcategories only within their valid date range.

4. **Migration Routing** – Use mapping tables to redirect discontinued categories (e.g., Food & Beverages ⟶ Home & Kitchen).

5. **Split/Merge Handling** – For Smartphones split (2024-07-01), aggregate Android + iOS to represent Smartphones post-split.

6. **Cross-Attribute Join** – Join Sustainable flag table on date range to track flag changes over time.

7. **Integrity Constraints** – No product may belong to two active categories at the same time; all migrations must be complete.

Example OLAP Query Scenarios

1. Quarterly Sales of Smartphones (Q1 2023 – Q2 2025)
Include Smartphones (before July 2024) and Android + iOS (after July 2024).

```
select
   concat(year(s.sale_date), '-q', quarter(s.sale_date)) as quarter,
   sum(s.revenue) as smartphone_sales
from sales s
join product_classification_history p
   on s.product_id = p.product_id
   and s.sale_date between p.eff_from and ifnull(p.eff_to, '9999-12-31')
join classification_dim c
   on p.classification_id = c.id
where c.name in ('smartphones', 'android phones', 'ios phones')
group by year(s.sale_date), quarter(s.sale_date)
order by year(s.sale_date), quarter(s.sale_date);
```

2. Android vs iOS Sales (after July 2024)

```
select
   date_format(s.sale_date, '%y-%m') as month,
   c.name as platform,
   sum(s.revenue) as total_sales
from sales s
join product_classification_history p
   on s.product_id = p.product_id
   and s.sale_date between p.eff_from and ifnull(p.eff_to, '9999-12-31')
join classification_dim c
```

```
   on p.classification_id = c.id
where s.sale_date >= '2024-07-01'
  and c.name in ('android phones', 'ios phones')
group by year(s.sale_date), month(s.sale_date), c.name
order by year(s.sale_date), month(s.sale_date);
```

3. Children's Products Growth (since Nov 2024)

```
select
   concat(year(s.sale_date), '-q', quarter(s.sale_date)) as quarter,
   sum(s.revenue) as child_products_sales
from sales s
join product_classification_history p
   on s.product_id = p.product_id
   and s.sale_date between p.eff_from and ifnull(p.eff_to, '9999-12-31')
join classification_dim c
   on p.classification_id = c.id
where c.parent_category = 'children''s products'
  and s.sale_date >= '2024-11-01'
group by year(s.sale_date), quarter(s.sale_date)
order by year(s.sale_date), quarter(s.sale_date);
```

4. Gaming Electronics Performance (2023 – 2025)

```
with gaming_products as (
   select distinct product_id
   from product_classification_history
   where classification_id = (
      select id from classification_dim where name = 'gaming electronics'
   )
)
select
   date_format(s.sale_date, '%y-%m') as month,
   sum(s.revenue) as gaming_sales
from sales s
where s.product_id in (select product_id from gaming_products)
group by year(s.sale_date), month(s.sale_date)
order by year(s.sale_date), month(s.sale_date);
```

5. Asian Regional Report (with Mobile Accessories)

```
select
   concat(year(s.sale_date), '-q', quarter(s.sale_date)) as quarter,
   c.name as subcategory,
   sum(s.revenue) as revenue
from sales s
join product_classification_history p
   on s.product_id = p.product_id
   and s.sale_date between p.eff_from and ifnull(p.eff_to, '9999-12-31')
join classification_dim c
   on p.classification_id = c.id
where s.region = 'asia'
  and c.region in ('asia', 'global')
group by year(s.sale_date), quarter(s.sale_date), c.name
order by year(s.sale_date), quarter(s.sale_date);
```

Data Integrity Constraints

1. Each product must have only one active classification per date and region.
2. No overlapping validity intervals in product_classification_history.
3. All migrations (Food & Beverages → Home & Kitchen, etc.) must cover 100% of affected products.
4. Temporary categories (Refurbished Devices) must expire without overlaps.
5. Cross-attribute flags (Sustainable) tracked independently with their own validity intervals.
6. Aggregated sales totals across categories must equal base sales table totals after any migration or split.

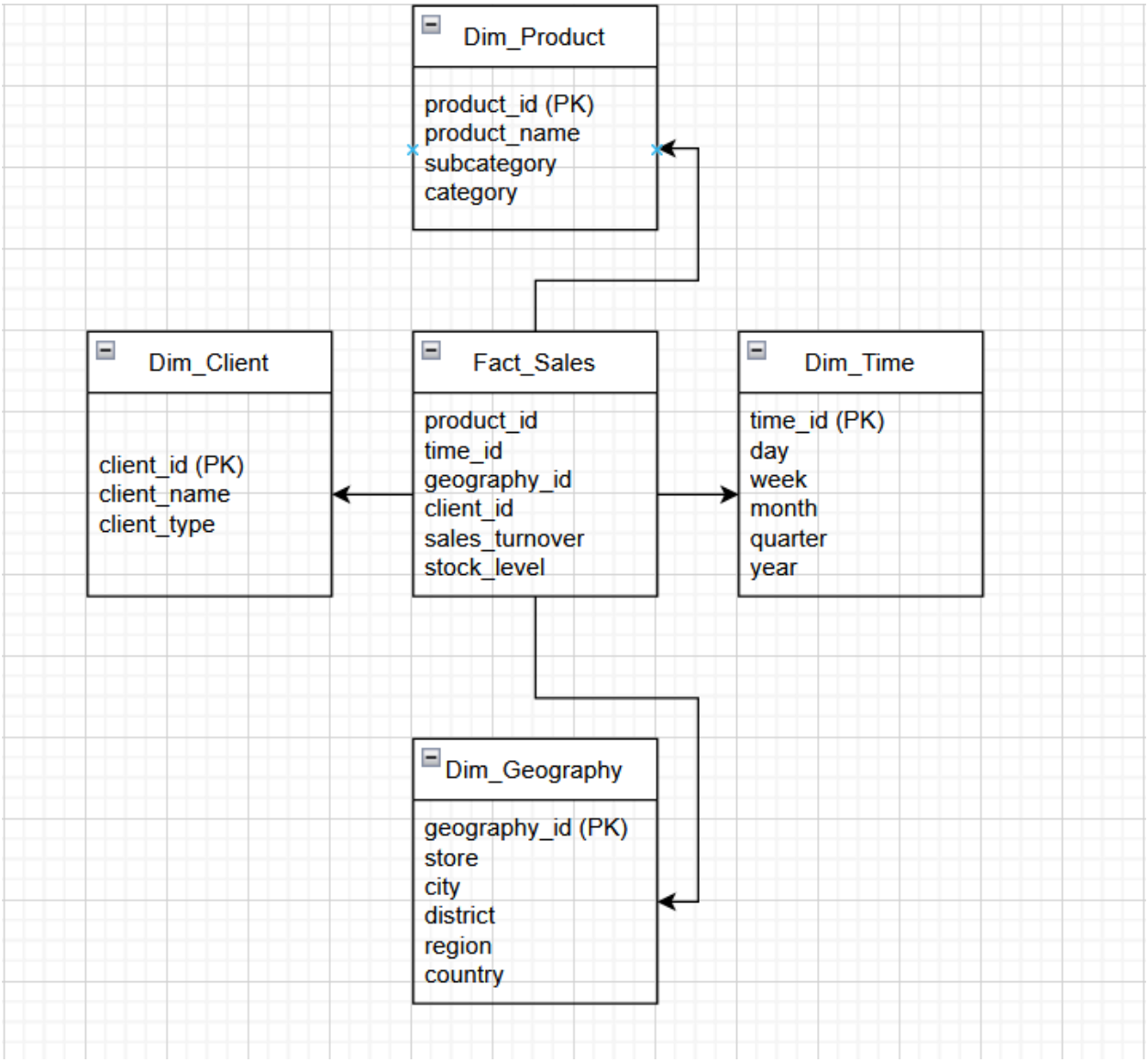
Result:

This design enables consistent temporal, regional, and business-rule-aware analytics across all

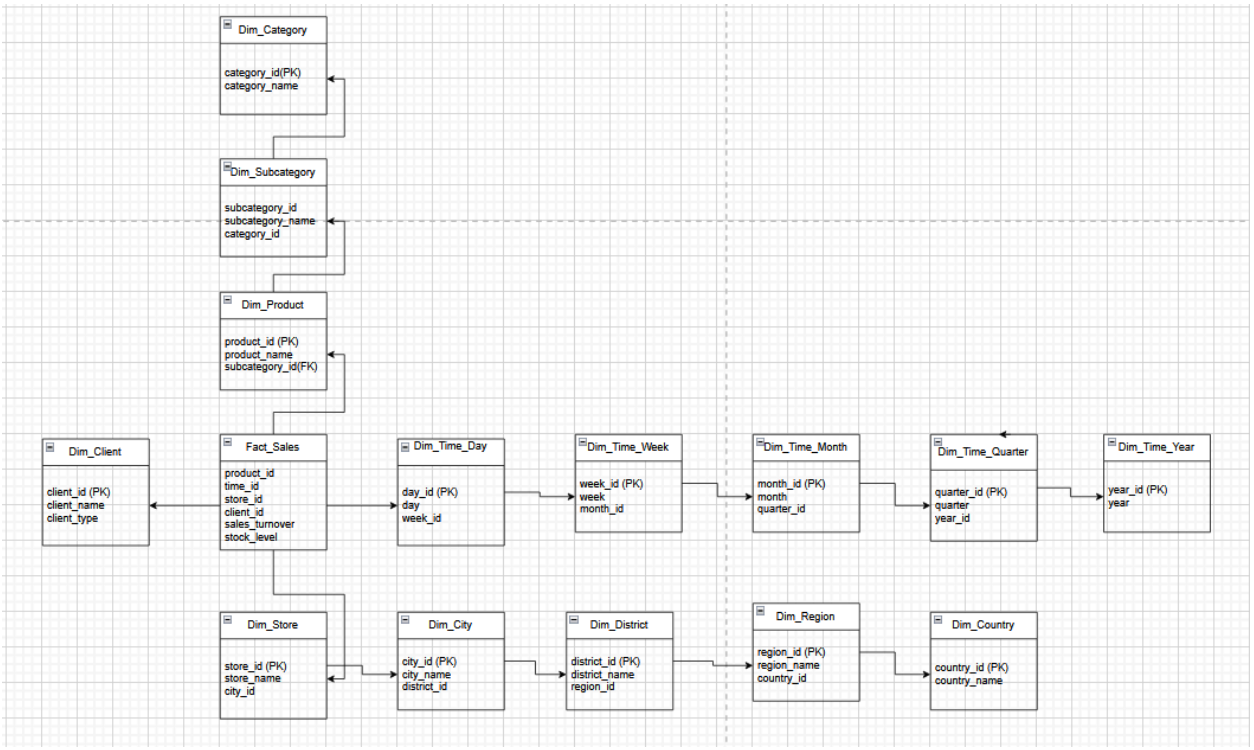
schema versions from 2023 to 2025, with full traceability for every classification change, ensuring correctness in OLAP queries and preventing double-counting or data gaps.

Q4:

1. Star Schema Design



2. Snowflake Schema Design



3. Schema Effectiveness & Justification

For retail sales analysis, the Star Schema is generally more effective because it allows faster query execution with fewer joins, making it ideal for OLAP operations like roll-ups and drill-downs. It also handles large-scale aggregations efficiently across dimensions such as Day, Store, Region, and Client Type. While the Snowflake Schema provides better normalization and reduces redundancy, its added complexity and extra joins make it slower for analytical queries. In conclusion, the Star Schema is recommended for its simplicity, performance, and overall suitability for retail sales data analysis.